

How job numbers, films and territories are resolved

The exact mechanics behind the guide — data sources, resolution order, and the rules that keep the three sources from disagreeing. Pairs with *job-data-guide.pdf*.

1 The identity model — everything keys on the XY code

Everything keys on the **XY code**, not the human-readable string. A job number arrives in several shapes and is normalised before it is ever used as a key:

```
// src/utils/wrikeyHelpers.js - jobKey()
"XY025716"                // bare code
"XY025716_LUG_D6"         // suffix → still XY025716
"The Odyssey : XY025716, Finishing" // canonical → still XY025716

const jobKey = (jobNumber) => {
  const m = String(jobNumber).match(/XY\d{5,6}/i);
  return m ? m[0].toUpperCase() : String(jobNumber).trim(); // fallback: free-text internal jobs
};
```

The same regex is a uniqueness index in the database:

```
-- supabase/schema.sql - jobs_job_code_key
create unique index jobs_job_code_key
on public.jobs (substring(job_number from 'XY\d{5,6}'));
```

So a bare code from an old row, a suffixed code from a folder, and the curated canonical string all collapse onto one job, and a job can never be registered twice under two different spellings.

The two stores

- **Job Book** — `jobs`. The curated, admin-editable record. Holds a canonical `"Film : XY..., Description"` string plus `film_title` and `client`. Loaded via `src/hooks/useJobLookup.js`.
- **Timesheet rows** — `tasks`. One row per entry, carrying `job_number`, `territory`, `film_title`, `client`, `source` (`"tracker"` | `"legacy"`).

2 Management — the Wrike scan (`src/lib/wrikeCampaign.js`)

`scanStudioJobNumbers()` walks the entire visible Wrike folder tree and returns one candidate Job Book row per unique XY code. Real job folders sit at `Studio space → <Studio> → <Film> → <Job>`.

FOR EVERY FOLDER WHOSE TITLE CARRIES AN XY CODE

1. Take the **code**.
2. Read the **description** from the folder-title suffix after the code (`_Germany_Launch_Assets` → "Germany Launch Assets") via `deUnderscore()`.
3. **Climb the ancestry** (`ancestryOf` → `describeChain`) to the studio folder. The studio's *keyword* (`studioKwOf`) names the client; the *deepest non-year folder* between the studio and the job is the **film** (year folders like `2026` are skipped — `isYearFolder`).
4. **Region marker** off the studio folder's full title (`regionOf(studioTitle, studioKw)`): "UNIVERSAL UK" → UK arm, plain "UNIVERSAL" → International, XYi → none. This decides the *client* ("Universal Pictures UK" / "Universal Pictures International") and the description prefix ("UK - ..."), not the country territory.
5. Assemble the canonical line: `"Film : CODE, Description"`. A folder already in canonical shape (title contains `:`) is taken verbatim.

Each candidate carries `code`, `jobNumber`, `filmTitle`, `projectDescription`, `client`, `archived`, `region`, `folderId`, `folderPath`. The `folderPath` breadcrumb (studio › film › folder) is computed from the winning ancestry chain and is what

the review panel shows next to a disagreement. Wrike's `createdDate` per folder is fetched in batches of 100 for ordering.

Multi-parent folders. `parentsOf` is a *list* map, not a single-parent map — a shared folder can sit under several paths at once. `ancestryOf` walks every path (bounded: depth 40, 24 chains) and ranks them: live-and-placed > placed > live > anything. A job is only “archived” when it has *no* live home, so one path through `_Archive` doesn't hide a job that also lives in a campaign folder.

Reconciliation with the Job Book

Candidates are matched against the book on the code. “Containment counts as agreement” — a book description and a scan description legitimately carry different amounts of the same information, so agreement is *contains*, not *equals*. Genuine disagreements surface in the Review panel as **corrections** (book row disagrees with Wrike) and **redundant rows** (several book rows collapse onto one code). Each row shows the book's line, the proposed Wrike line, and the Wrike folder path. **Fix** rewrites the book to match Wrike; **Keep** records the code in `job_sync_kept` (`supabase/migrations/20260809220000_job_sync_kept.sql`) and future scans skip it.

3 Legacy — resolving a pulled task

Legacy rows come from Wrike timelogs (or `handleSyncMyJobs`). Each task is resolved through `guessFieldsFromTask()` (`src/utils/wrikeHelpers.js`; the component keeps a copy tuned for its data shape) in this order:

RESOLUTION ORDER FOR ONE TASK

1. **Job number** — the dedicated “Job Number” custom field first (raw value, suffix preserved so `XY025716_LUG_D6` survives), else the first `XY\d{5,6}` in the task title/description/path. The bare code is then fed to `resolveJobNumber(fullCode, bookJobNumbers)`.

2. **Job Book match** — `resolveJobNumber` finds the book row whose string contains the code and returns that string *whole*. The book wins; anything the suffix might add belongs in the description, not the code. Unknown codes return the **bare** code (never the suffixed one) so the Studio Scan can fill them in.

3. **Territory (country)** — `resolveCountries(task, parentTaskName, ...)`, see §4. Multiple countries join via `joinTerritories()` into e.g. “UK, Ireland”.

4. **Film** — priority: the job number's own “Film : ...” prefix (split on `:` specifically, so a film title containing a colon is safe) → the Job Book's `film_title` → `task.projectName` / path-derived title (most-frequent folder before `DIGITAL` across paths, to dodge reference trees) → the task title's first token → `FILM_MAPPINGS` overrides. All-caps names are title-cased.

5. **Client** — path contains UNIVERSAL/PARAMOUNT/SONY → client from studio + resolved territory (“Universal Pictures UK” if UK, “Universal Pictures Australia” if Australia, else “Universal Pictures International”).

6. **Job Book override** — `jobLookup.getJob(code)`: a known book row replaces *any* guess above for `film_title` and `client`, and upgrades a bare code to the book's canonical string.

7. **Self-registration** — `jobLookup.ensureJob(number, { filmTitle, client })` inserts the row verbatim the first time the code is seen, or fills only blank fields on an existing row. Never overwrites a set field; ignores the `23505` duplicate-violation (another tab already wrote it).

4 Territory resolution — read, never guessed

The old rule scanned every scrap of text on a task (title, path, notes, custom fields) and kept whichever country matched first — and since 44 of the ~120 short codes are two letters, it read countries out of plain prose: “ODYSSEY_IN_PROGRESS” → India, “No changes needed” → Norway. That was removed. A country code is now only read *where someone deliberately put one*, in priority order (`src/utils/countryCodes.js`, `src/lib/countryField.js`):

Order	Source	Notes
1	Last token of the task name	Anchored — <code>...IT</code> resolves; <code>...IT_Deck</code> does not.
2	Last token of the parent task name	Only when the task itself has no suffix.
3	Folder tree climb (<code>getFolderCountries</code>)	Nearest folder first, whole level before climbing; bounded depth.
4	Country custom field	Matched by field <i>id</i> (<code>countryFieldIds()</code>), never by the shape of its value.

The alias table is ordered weakest-first so real codes can't be shadowed: territory names < `REGION_ALIASES` < `MAGI_MARKET_CODES` (what production actually writes) < agreed exceptions (`COUNTRY_SUFFIX_EXCEPTIONS`). Keys are punctuation-stripped (`codeKey`), so MAGI's `BE-FL` arrives as `BEFL` however it was typed. A task that deliberately carries no country now gets *no* country rather than a guessed one — the row pulls and the field is left for a human.

Territory vs. region marker. "Territory" here — the `tasks.territory` column, what `resolveCountries` returns — means the **country** (UK, Ireland, "UK, Ireland"). The INT/UK/NM/AUS short codes from `regionOf()` and `REGION_SHORT` are studio region markers that shape the client name and description prefix; they are a different concept and shouldn't be called territory.

5 Where the pieces live

Concern	File	Responsibility
Job lookup / registration	<code>src/hooks/useJobLookup.js</code>	Loads <code>jobs</code> keyed by code; <code>getJob</code> , <code>ensureJob</code> , <code>jobNumbers</code> .
Wrike scan / film & client derivation	<code>src/lib/wrikeCampaign.js</code>	<code>scanStudioJobNumbers</code> , <code>ancestryOf</code> , <code>describeChain</code> , <code>regionOf</code> .
Task → fields guess	<code>src/utils/wrikeHelpers.js</code>	<code>jobKey</code> , <code>resolveJobNumber</code> , <code>guessFieldsFromTask</code> .
Territory (country) resolution	<code>src/utils/countryCodes.js</code> + <code>src/lib/countryField.js</code>	<code>resolveCountries</code> , <code>countriesFromTaskName</code> , <code>countriesFromCustomFields</code> .
Folder → countries / description	<code>src/lib/wrikeEnrich.js</code>	<code>getFolderCountries</code> , <code>jobFolderDescription</code> , <code>buildChildToParent</code> .
Multi-country join	<code>src/utils/territories.js</code>	<code>joinTerritories</code> / <code>splitTerritories</code> .
Legacy timesheet UI / pull	<code>src/components/LegacyTimesheets.js</code> , <code>src/hooks/useLegacyRows.js</code>	Pulls timelogs, runs <code>guessFieldsFromTask</code> , writes rows via <code>useTasks</code> scoped to <code>source="legacy"</code> .
Management UI / scan review	<code>src/components/Management.jsx</code>	Film picker (grouped by studio), Job Book, <code>StudioJobScanModal</code> with Fix/Keep.
Keep table	<code>supabase/migrations/20260809220000_job_sync_kept.sql</code>	XY codes whose disagreements the scan should stop re-asking.

Data model (relevant columns)

Table	Columns
<code>jobs</code>	<code>id</code> , <code>job_number</code> , <code>client</code> , <code>film_title</code> , <code>project_description</code> , <code>wrike_folder_id</code> , <code>wrike_folder_title</code> , <code>start_date</code> , <code>completed_date</code> , <code>status</code> , <code>notes</code> , <code>created_at</code> , <code>updated_at</code>
<code>tasks</code>	<code>id</code> , <code>job_number</code> , <code>territory</code> , <code>category</code> , <code>notes</code> , <code>date</code> , <code>work_date</code> , <code>film_title</code> , <code>client</code> , <code>project_description</code> , <code>time_spent</code> , <code>additional_time</code> , <code>source</code> , <code>wrike_user_id</code> , <code>task_id</code> , <code>wrike_timelog_id</code>
<code>films</code>	<code>id</code> , <code>title</code> , <code>studio</code> (studio added for the grouped picker; backfilled from the client most of the film's jobs bill to)
<code>job_sync_kept</code>	<code>code</code> , <code>created_at</code>

Engineering notes for the Timesheetter app · Describes the exact resolution of job numbers, films and territories for tasks in Management and Legacy. For the plain-English version see `job-data-guide.pdf`.